the server-side rate limiter pattern, and this section will show you how to use it for Edge Service by using Spring Cloud Gateway and Spring Data Redis Reactive. Let's start with setting up a Redis container.

9.3.1 Running Redis as a container

Imagine you want to limit access to your API so that each user can only perform 10 requests per second. Implementing such a requirement would require a storage mechanism to track the number of requests each user performs every second. When the limit is reached, the following requests should be rejected. When the second is over, each user can perform 10 more requests within the next second. The data used by the rate-limiting algorithm is small and temporary, so you might think of saving it in memory inside the application itself.

However, that would make the application stateful and lead to errors, since each application instance would limit requests based on a partial data set. It would mean letting users perform 10 requests per second per instance rather than overall, because each instance would only keep track of its own incoming requests. The solution is to use a dedicated data service to store the rate-limiting state and make it available to all the application replicas. Enter Redis.

Redis (https://redis.com) is an in-memory store that is commonly used as a cache, message broker, or database. In Edge Service, we'll use it as the data service backing the request rate limiter implementation provided by Spring Cloud Gateway. The Spring Data Redis Reactive project provides the integration between a Spring Boot application and Redis.

Let's first define a Redis container. Open the docker-compose.yml file you created in your `polar-deployment` repository. (If you haven't